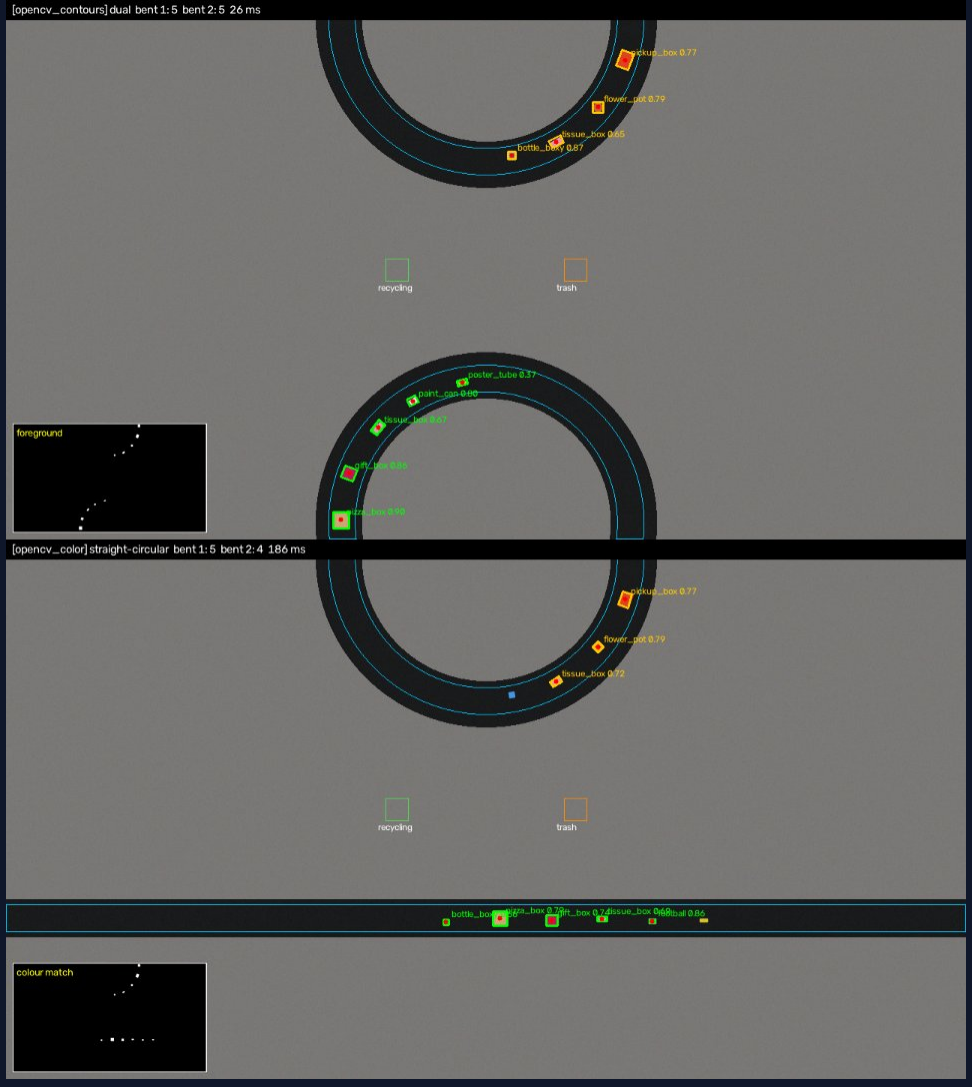


REPORT 06 · FACTORY BENT SIMULATION WITH A BCR ROBOT ARM

Vision Lab & Testing

OpenCV / YOLO development environment, labs, pipeline demo and test results

ROS 2 Jazzy · Gazebo Harmonic · OpenCV · PyQt5 · BCR 7-axis arm

Hüseyin Tozluyurt · November 2026

1. Why a vision lab

By Oct 31 the arm worked, but the detector was built into one node. The project's goal is to develop **object detection algorithms** and see them drive a robot. The vision lab (Oct 31, 18:00) separates the two: you write a detector and a `main(lab)` function, and the lab provides the camera, coordinate conversion, controller, arm, belts, bins and scoring in any world.

```
./run_vision_lab.sh sim --world dual        # terminal 1: simulation without the built-in detector/controller
./run_vision_lab.sh run 04_sort_by_label    # terminal 2: any lab file
./run_vision_lab.sh new my_lab --template sort
./run_vision_lab.sh offline --synthetic 20 --world all   # score detectors without Gazebo
```

2. The API

Call	Does
`lab.detect_objects(bent=1)`	detections in world coordinates: label, score, x, y, yaw, size, bent
`lab.move_to_bent(2)`, `lab.move_to_trash()`, `lab.move_to_recycle_bin()`	the next product at the station goes there
`lab.run(decide, count=N)`	keep catching. `decide(ctx)` returns bent2 / trash / recycling / skip
`lab.run({"paint_can": "recycling", "*": "bent2"})`	the same rule as a table keyed by label
`lab.spawn_products([...])`, `lab.stop_bent(n)`, `lab.set_motion(...)`	belts, orders and arm style
`lab.snapshot()`, `lab.start_recording(dir)`	annotated images, auto-labelled YOLO dataset
`lab.report()`	where each product went and whether the detector's label matched the simulation's truth

3. Ten labs

Lab	Experiment
01 detection_survey	detection only: counts per bent and label, snapshots
02 bent_to_bent	bent 1 → bent 2 (recycling in worlds without bent 2)
03 trash_and_recycle	products into both bins
04 sort_by_label	a decide(ctx) rule on the detector's label
05 my_opencv_detector	your own OpenCV detector class inside the lab file
06 yolo_sorting	YOLO labels choose the destination
07 record_dataset	an auto-labelled YOLO dataset while the arm works
08 pipeline_sorting	all 12 pipeline stations sort by vision
09 pipeline_demo	three acts: detection at A1, arm A1, the whole factory. Snapshots, video, report
10 pipeline_tests	8 test cases with PASS/FAIL for each product

4. The pipeline lab

`pipeline_sim.py` moves belts, products and arms kinematically at 20 Hz over gz-transport, so 12 arms run without ROS controllers. It also has a `--fake` backend that draws the camera images, so labs and tests run without Gazebo. While building it, one bug in the layout was found. `pipeline_layout.py` says all rings turn counter-clockwise, which would hand every product on C1 and C2 to the "all" arm before the selective arm sees it. The lab therefore runs C1 and C2 clockwise.



Act 1: detection at station A1



Act 2: after arm A1

Demo report (lab 09, `--fake` backend)

```
16 decisions: A1:bent2 6, A1:recycling 1, A1:trash 1, A2:bent2 1, A2:skipped 1, A3:bent2 2, A4:bent2 1, A5:bent2 2, A
detector label correct (vs simulation truth): 16/16 = 100%
where products are now: S1 4, C1 3, C2 1, C3 2, trash_bin_A1 1, recycling_bin_A1 1
  A1 #1: delivered -> bent2     label=cereal_box truth=cereal_box [ok]
  A1 #2: delivered -> trash     label=glass_jar truth=glass_jar [ok]
  A1 #3: delivered -> recycling label=sneaker_pair truth=sneaker_pair [ok]
  A1 #4: delivered -> bent2     label=shoe_box truth=shoe_box [ok]
  A3 #1: delivered -> bent2     label=cereal_box truth=cereal_box [ok]
  A1 #5: delivered -> bent2     label=coffee_can truth=coffee_can [ok]
  A5 #1: delivered -> bent2     label=cereal_box truth=cereal_box [ok]
  A2 #1: skipped   -> skip      label=basketball truth=basketball [ok]
  A8 #1: missed    -> bent2     label=cereal_box truth=cereal_box [ok]
  A1 #6: delivered -> bent2     label=basketball truth=basketball [ok]
  A1 #7: delivered -> bent2     label=pizza_box truth=pizza_box [ok]
  A2 #2: delivered -> bent2     label=coffee_can truth=coffee_can [ok]
  A3 #2: delivered -> bent2     label=shoe_box truth=shoe_box [ok]
  A1 #8: delivered -> bent2     label=tool_case truth=tool_case [ok]
  A4 #1: delivered -> bent2     label=coffee_can truth=coffee_can [ok]
  A5 #2: delivered -> bent2     label=shoe_box truth=shoe_box [ok]
```

5. Pipeline test cases

`pipeline_testing.py` puts known products on S1, lets a chosen set of stations work with a routing table, works out each product's expected route from the same table applied to its **true** type, and reports PASS or FAIL for each product. A failure says why: a wrong label, a missed grasp, a fall, or a late arrival.

Case	Route	Result (--fake run, Nov 1)
one_hop_A1	S1 → C1	PASS (3/3), 96 s
bins_at_A1	A1 into its trash bin, its recycling bin and C1	PASS (3/3), 84 s
two_hops_to_S3	S1 → C1 → S3	PASS (2/2), 119 s
chain_to_large_dock	S1 → C1 → S3 → C3 → S7 → dock 3	PASS (2/2), 244 s
branch_on_C1	A2 → dock 2, A3 → dock 3	PASS, 328 s

Case	Route	Result (--fake run, Nov 1)
bins_along_the_chain	A3 recycles pizza boxes, A5 trashes coffee cans, rest → dock 3	PASS, 299 s
longest_route_to_S9	6 hops: S1 → C1 → S3 → C3 → S6 → C4 → S9 → dock 5	PASS, 307 s
whole_factory_rules	all 12 stations (stress test)	not in this run (1–2 wrong docks possible on a small PC)

All 7 cases in the logged run passed, but the detector's labels were right in only 48 of 60 decisions (80 %). Most of the misses are `shoe_box` labelled as `cookie_tin` (marked [WRONG] in pipeline_test_run.txt), two types with similar top colours. Those products still took the route their case expected. This is the kind of finding the lab is built to show, and a good first target for a trained YOLO model.

6. Testing on the real PC

On Nov 1 the workspace was built with colcon on the user's computer, and the straight conveyor world was run. All 7 products spawned and rode the belt steadily (about 3.9 m in 90 s), and the user confirmed it worked. The other worlds and the labs were not run in that session, so their Gazebo behaviour is still marked "not tested in Gazebo yet" in COMMANDS.md.

7. Next steps

- Run the remaining worlds and the pipeline lab in Gazebo on the real PC. Confirm the station cameras and `set_pose_vector` arm posing.
- Train YOLO on a lab 07 dataset and compare it with `opencv_contours` in lab 06 / offline scoring (shoe_box vs cookie_tin).
- Spawn one custom `.dae` product in Gazebo to confirm its unit and Y-up handling.
- Commit the work to the git repository (most of `factory_bringup/` is still uncommitted).